

2.1)

- 1) Según la línea de código que Chat GPT consideraba incorrecta, es verdad que la presencia de "R1 != R2" es necesaria para restringir correctamente el funcionamiento del código, sin embargo no entrega la solución más óptima. En cambio, la solución que utiliza "S1 >= S2." si aplica una restricción que reduce el número de reglas y/o literales innecesarios.
- 2) Chat GPT tiene razón, pues solo limita el máximo, sin mínimo, sin embargo, al plantear su solución se equivoca, colocando entre 4 y 6 ramos, siendo que por enunciado hay que asegurarse *"de tomar entre 10 y 50 créditos (o entre uno y cinco ramos) cada semestre"*, por lo que se debe colocar un 1 en vez de un 4 o un 0.
- 3) Es incorrecto, pues el operador "\ " si existe y es el que define el operador de módulo, donde #mod o "mod " no existen en la sintaxis de clingo. Sin embargo, se puede optimizar la solución entregada por DeepSeek ya que entrega los literales "par(R), ramo(R) y semestre(S)" que son redundantes y se pueden evitar utilizando únicamente el operador "\ " para R (ramo) y S (semestre).
- 4) La sintaxis "not 1 {...}" es completamente válida y se utiliza constantemente en restricciones (por ejemplo, para evitar que algo sea verdadero al mismo tiempo que lo es una restricción de cardinalidad) , sin embargo la solución simplificada que se entrega fue la que se aplicó en (1), ya que esta es más elegante y eficiente. Es decir, a pesar de que la justificación es incorrecta, su propuesta es la más óptima.
- 5) Como mencioné en (2) es verdad que había redundancia en la definición de literales, sin embargo estos fueron descartados con mi mejora de la solución entregada por DeepSeek.

2.2)

Mi versión de la documentación fue un poco más extensa (dentro de los límites establecidos por al pauta) que la generada con IA, esto se puede deber a que en mi prompt le dije que su respuesta debía ser *"breve, concisa y fácil de entender para cualquier persona"*, lo que se tomó muy en serio y definió todo en máximo 1 línea. El problema yace en que esa sintetización quita cierta información que también puede ser necesaria, es decir, ChatGPT asume que ya entendemos el contexto del código, lo cual no es cierto para todos los casos. Por ejemplo en el ítem IV de su respuesta habla de ramos pares, lo cual yo expliqué de forma más detallada, para que se entendiera bajo cualquier contexto.

En general, opino que la versión de la IA es más directa y no gasta su tiempo en explicar cosas del enunciado, mientras que mi versión usa más caracteres para tratar de explicar cosas "más

obvias”, como por ejemplo el mismo funcionamiento de la lógica de una regla. Además, uso un lenguaje más informal, manteniendo la correctitud en la explicación.

Por mi parte, creo que pude mejorar mi capacidad de síntesis, pues para todas las reglas usé 2 líneas para la descripción. Por otro lado, ChatGPT pudo haberse explayado un poco más en sus descripciones, para que cualquier usuario pueda comprender el código.

2.3)

Mi solución tuvo en consideración las restricciones y reglas anteriores, para así no generar un programa redundante, la más importante fue cambiar la regla “1 {...} 5 :-” por “0 {...} 5 :-”, para que se generen soluciones no triviales ($\neq \text{max_semestre}(14)$), también no necesité utilizar herramientas que no conozco del todo bien, como “M = #max {...}” ni tampoco editar reglas que ya estaban establecidas en el programa, a diferencia de la IA.

Por otro lado, viendo las similitudes tenemos que si se cambia regla “1 {...} 5 :-” por “0 {...} 5 :-” en *parte3_gpt.lp*, se obtiene el mismo resultado en ambos programas: *max_semestre(9)*, esto demuestra que hay diferentes formas de llegar a los mismos resultados.

Como cosas a mejorar, sé que mi código no es el más óptimo, pues entrega más de 1 modelo (4 exactamente) y entiendo que tiene que ver con la presencia del *#maximize* y el *#minimize* a la vez, donde se debería usar algún tipo de directiva para establecer un orden de importancia. Por otro lado, la IA no es muy amigable para el *debuggeo*, además se siente demasiado artificial su respuesta, donde se nota mucho que está respondiendo a un prompt.

Archivos Adjuntos: Conversación con ChatGPT para 2.2

Hola ChatGPT!! Cómo estás? Nuevamente estoy realizando una tarea para el ramo "Inteligencia Artificial". Esta vez me pidieron que te escribiera para que realices una documentación de un programa con extensión .lp (clingo), esta debe ser breve, concisa y fácil de entender para cualquier persona. A continuación te entregaré el código, donde las reglas/restricciones que tienes que comentar son las que tienen los número romanos (I - IV). Te envío todo el código para que tengas el contexto, pero no es necesario que me reenvíes todo de nuevo, si quieres me puedes entregar las 4 aclaraciones por separado para que yo las pueda copiar y pegar en mi documento, está todo claro para que podamos seguir con nuestra conversación?

¡Hola Martín! 😊 Estoy muy bien, gracias por preguntar. Entiendo perfectamente lo que necesitas: me darás un código en **Clingo (.lp)** y yo debo hacer una documentación breve, concisa y clara de las reglas/restricciones que están marcadas con números romanos (I – IV).

No es necesario que yo copie todo el código de vuelta, sino que te entregue **4 explicaciones independientes** para que luego puedas copiarlas en tu documento.

¿Quieres que las haga en un estilo más **académico/técnico** (tipo informe universitario) o en un estilo más **didáctico/explicativo** para que alguien sin experiencia también lo entienda?

📄 🗑️ 🔍 ↕️ ↻️ ⋮

Deben ser máximo 2 líneas de extensión (siguiendo las normas del PEP8 y siendo comentadas con el símbolo "%").

Perfecto 😊. Entonces cuando me pases el código en .lp, te redacto los 4 comentarios (I–IV), cada uno con máximo 2 líneas, empezando con %, claros y fáciles de entender.

¿Quieres que te los entregue en el mismo orden (I, II, III, IV) listos para copiar y pegar en tu archivo?

📄 🗑️ 🔍 ↕️ ↻️ ⋮

```
% ===== Hechos base (NO MODIFICAR) =====
ramo(1..23).

semestre(1..14).

% ===== Relaciones posibles (NO MODIFICAR) =====
requisito(1,6). requisito(1,7).
requisito(2,6). requisito(2,8).
requisito(3,7). requisito(3,9).
requisito(4,8). requisito(4,9).
requisito(5,10).

requisito(6,11).
requisito(7,14).
requisito(8,12).
requisito(9,13).
requisito(10,15).

requisito(11,16). requisito(12,16).
requisito(13,17). requisito(14,17).
requisito(15,18).

requisito(16, 21). requisito(17, 21).
requisito(18, 22). requisito(19, 23).

ramo_semestre(1, 1).

par(6).
impar(1).
```

+ Pregunta lo que quieras 🔍 🗑️

```
par(6).
impar(1).

% ===== Código Arreglado (Para ChatGPT) =====

% I
1 {ramo_semestre(R, S) : ramo(R)} 5 :- semestre(S).

% II
{ramo_semestre(R, S) : semestre(S)} = 1 :- ramo(R).

% III
:- requisito(R1, R2), ramo_semestre(R1, S1), ramo_semestre(R2, S2),
S1 >= S2.

% IV
:- ramo_semestre(R, S), S \ 2 == 1, R \ 2 == 0.
:- ramo_semestre(R, S), S \ 2 == 0, R \ 2 == 1.

% ===== IGNORAR =====

superior_semestre(S) :- ramo_semestre(_, S2), S2 > S, semestre(S).

#maximize { R : ramo_semestre(R, _) }.

#show ramo_semestre/2.
```

¡Perfecto Martín! Te dejo las explicaciones para cada restricción/regla I–IV, listas para copiar/pegar en tu archivo `.lp`.

¡Perfecto Martín! Te dejo las explicaciones para cada restricción/regla I–IV, listas para copiar/pegar en tu archivo `.lp`.

I

prolog

Copiar código

% Limita a que en cada semestre se cursen entre 1 y 5 ramos.

II

prolog

Copiar código

% Obliga a que cada ramo se dicte exactamente en un semestre.

III

prolog

Copiar código

% Asegura que un ramo se curse después de cumplir sus requisitos previos.

IV

prolog

Copiar código

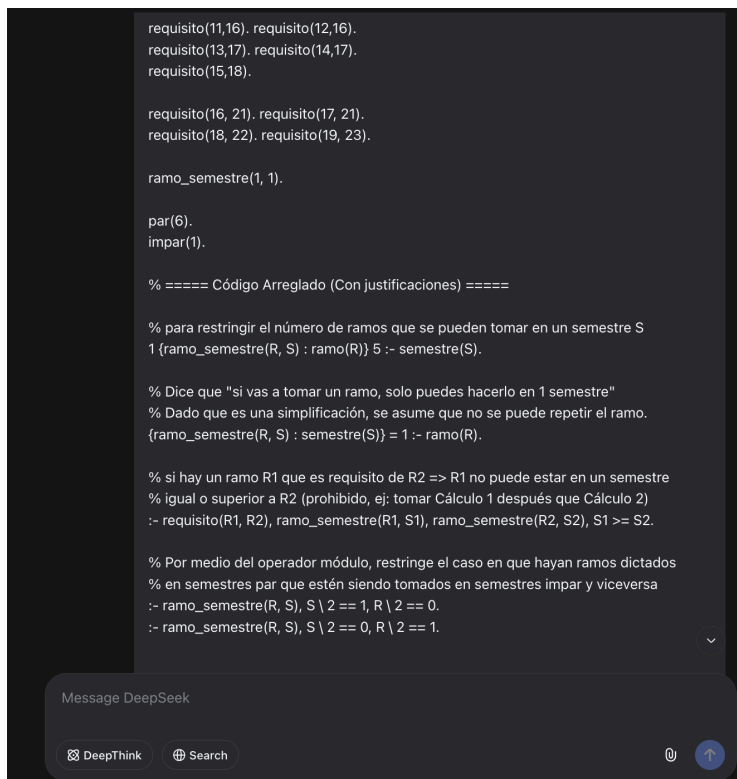
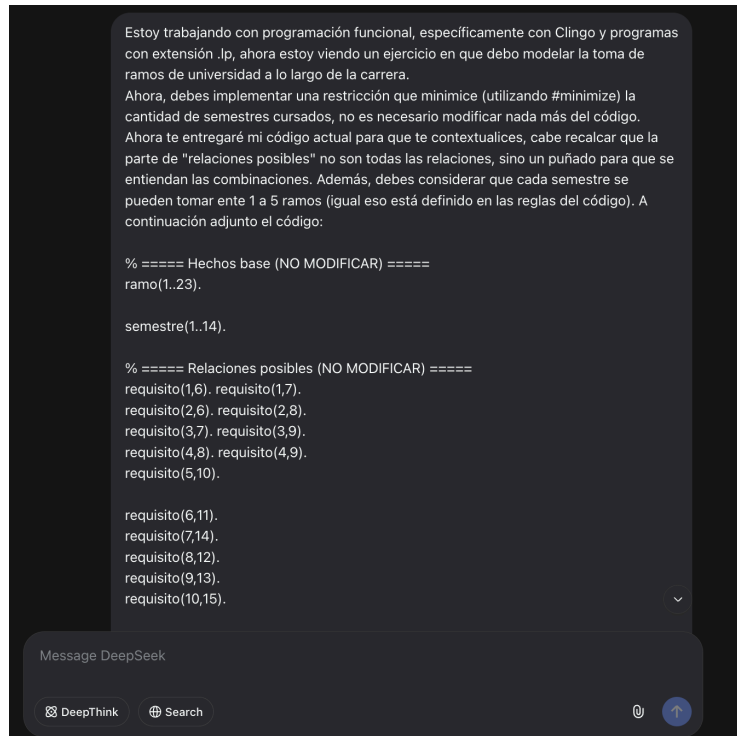
% Restringe que ramos pares vayan en semestres pares e impares en impares.

¿Quieres que también te prepare una versión un poco más extendida (3–4 líneas) como respaldo por si el profe pide más detalle, o prefieres dejarlo así bien conciso?

📄 👍 🗨️ ↗️ ↺️ ⋮

LINK DE CONVERSACIÓN: <https://chatgpt.com/share/68bf10ba-f788-8004-bd81-5596f27a4c2a>
(por si acaso no se ve muy bien en las capturas de pantalla)

Archivos Adjuntos: Conversación con DeepSeek para 2.3



```
#maximize { R : ramo_semestre(R, _) }.

% Aquí puedes definir la regla max_semestre con su minimización

% ===== Salida =====
#show ramo_semestre/2.
```

Para minimizar la cantidad de semestres cursados, necesitas agregar una restricción que penalice el uso de semestres. Aquí está la implementación:

```
clingo

% ===== Hechos base (NO MODIFICAR) =====
ramo(1..23).

semestre(1..14).

% ===== Relaciones posibles (NO MODIFICAR) =====
requisito(1,6). requisito(1,7).
requisito(2,6). requisito(2,8).
requisito(3,7). requisito(3,9).
requisito(4,8). requisito(4,9).
requisito(5,10).

requisito(6,11).
requisito(7,14).
requisito(8,12).
requisito(9,13).
requisito(10,15).
```

Explicación de la restricción agregada:

```
clingo

#minimize { 1,S : ramo_semestre(_, S) }.
```

Esta línea implementa la minimización de semestres de la siguiente manera:

- `ramo_semestre(_, S)` encuentra todos los semestres `S` que tienen al menos un ramo asignado
- `1,S` asigna un peso de 1 por cada semestre utilizado
- `#minimize` busca la solución que minimice la suma total de estos pesos

La restricción funciona penalizando cada semestre que contenga al menos un ramo, incentivando al solver a concentrar todos los ramos en la menor cantidad posible de semestres, respetando las demás restricciones del programa (como los requisitos, límites de ramos por semestre, etc.).

Okay, buena intuición, pero se me olvidó agregar un par de detalles muy importantes. Primero, el enunciado dice "Se debe procurar minimizar la cantidad de semestres para titularse, agregando un predicado "max_semestre", que defina el último semestre con ramos.", segundo, considera que se deben cursar todos los ramos para poder titularse siempre y cuando se cumplan las reglas, porque sino se da el caso en que tomas 1 ramo cada semestre y listo, o sino, tomas 23 ramos y listo.

Entiendo. Necesitamos asegurar que se tomen **todos los ramos** (los 23) y luego minimizar el semestre máximo en el que se cursa algún ramo. Aquí está la implementación corregida:

aquí corté el código porque repetía lo mismo (la parte de NO MODIFICAR)

clingoCopyDownload

```
ramo_semestre(1, 1).

par(6).
impar(1).

% ===== Código Arreglado (Con justificaciones) =====

% para restringir el número de ramos que se pueden tomar en un semestre S
1 {ramo_semestre(R, S) : ramo(R)} 5 :- semestre(S).

% Asegurar que TODOS los ramos se tomen exactamente una vez
:- ramo(R), not ramo_semestre(R, _).

% si hay un ramo R1 que es requisito de R2 => R1 no puede estar en un semestre
% igual o superior a R2 (prohibido, ej: tomar Cálculo 1 después que Cálculo 2)
:- requisito(R1, R2), ramo_semestre(R1, S1), ramo_semestre(R2, S2), S1 >= S2.

% Por medio del operador módulo, restringe el caso en que hayan ramos dictados
% en semestres par que estén siendo tomados en semestres impar y viceversa
:- ramo_semestre(R, S), S \ 2 == 1, R \ 2 == 0.
:- ramo_semestre(R, S), S \ 2 == 0, R \ 2 == 1.

% Definir el último semestre con ramos (semestre máximo)
max_semestre(M) :- M = #max { S : ramo_semestre(_, S) }.

% Minimizar el semestre máximo (último semestre con ramos)
#minimize { M : max_semestre(M) }.

% ===== Salida =====
#show ramo_semestre/2.
#show max_semestre/1.
```

Message DeepSeek

Explicación de los cambios:

1. Asegurar que se tomen todos los ramos:

clingoCopyDownload

```
:- ramo(R), not ramo_semestre(R, _).
```

Esta restricción elimina cualquier solución donde algún ramo no sea tomado.

2. Definir el semestre máximo:

clingoCopyDownload

```
max_semestre(M) :- M = #max { S : ramo_semestre(_, S) }.
```

Este predicado encuentra el número de semestre más alto donde se toma algún ramo.

3. Minimizar el semestre máximo:

clingoCopyDownload

```
#minimize { M : max_semestre(M) }.
```

Esto minimiza el valor del último semestre, forzando a que todos los ramos se completen lo antes posible.

3. Minimizar el semestre máximo:

clingo

 Copy

 Download

```
#minimize { M : max_semestre(M) }.
```

Esto minimiza el valor del último semestre, forzando a que todos los ramos se completen lo antes posible.

Resultado: El programa ahora encontrará la distribución de ramos que permite completar todos los 23 ramos en la menor cantidad de semestres posibles, respetando:

- Los requisitos entre ramos
- Los límites de 1-5 ramos por semestre
- Las restricciones de paridad semestre/ramo
- La obligación de tomar todos los ramos para titularse

